

GET and POST Requests

The two main types of requests you'll use when building apps are GET and POST. You use *GET* requests for pages that only read data from the server. You usually use *POST* requests when the user needs to submit information through a form. We'll be specifying the POST method for processing all of our forms. (A few other kinds of requests exist, but we won't use them in this project.)

The `new_topic()` function takes in the request object as a parameter. When the user initially requests this page, their browser will send a GET request. Once the user has filled out and submitted the form, their browser will submit a POST request. Depending on the request, we'll know whether the user is requesting a blank form (GET) or asking us to process a completed form (POST).

We use an if test to determine whether the request method is GET or POST ❶. If the request method isn't POST, the request is probably GET, so we need to return a blank form. (If it's another kind of request, it's still safe to return a blank form.) We make an instance of `TopicForm` ❷, assign it to the variable `form`, and send the form to the template in the context dictionary ❸. Because we included no arguments when instantiating `TopicForm`, Django creates a blank form that the user can fill out.

If the request method is POST, the else block runs and processes the data submitted in the form. We make an instance of `TopicForm` ❹ and pass it the data entered by the user, which is assigned to `request.POST`. The form object that's returned contains the information submitted by the user.

We can't save the submitted information in the database until we've checked that it's valid ❺. The `is_valid()` method checks that all required fields have been filled in (all fields in a form are required by default) and that the data entered matches the field types expected—for example, that the length of text is less than 200 characters, as we specified in `models.py` in Chapter 18. This automatic validation saves us a lot of work. If everything is valid, we can call `save()` ❻, which writes the data from the form to the database.

Once we've saved the data, we can leave this page. The `redirect()` function takes in the name of a view and redirects the user to the page associated with that view. Here we use `redirect()` to redirect the user's browser to the topics page ❼, where the user should see the topic they just entered in the list of topics.

The context variable is defined at the end of the view function, and the page is rendered using the template `new_topic.html`, which we'll create next. This code is placed outside of any if block; it will run if a blank form was created, and it will run if a submitted form is determined to be invalid. An invalid form will include some default error messages to help the user submit acceptable data.